



Graph Neural Networks

안재국 / 2026.05.21



Computational Data Science LAB



CONTENTS



1. Why Graphs?
 2. Graph Basics & Prediction Tasks
 3. Permutation Invariance & Equivariance
 4. Neural Message Passing
 5. Graph Attention Networks
 6. Prediction with GNN & Challenges
- 
- 

01 | Why Graphs?

Why Graph-Structured Data Matters

- Basic idea

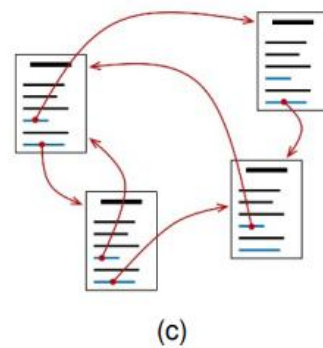
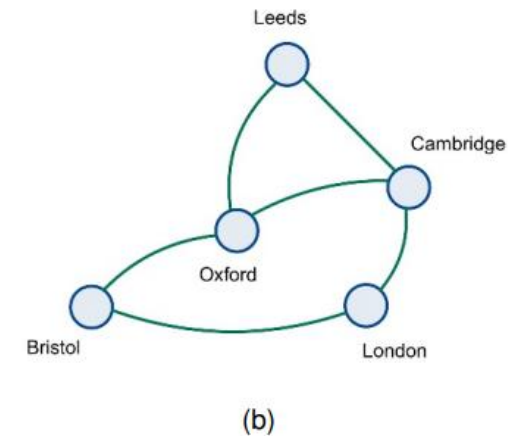
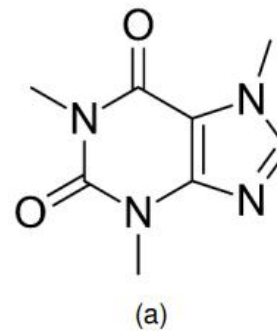
- ✓ 기존 딥러닝은 주로 sequence, image처럼 정해진 구조의 데이터를 다룸
- ✓ 하지만 현실 데이터는 객체 간 관계가 중요한 경우가 많음
- ✓ 그래프는 node와 edge로 “대상”과 “관계”를 함께 표현 가능

- Examples

- ✓ Molecule: atom = node, chemical bond = edge
- ✓ Rail network: city = node, railway line = edge
- ✓ Web graph: webpage = node, hyperlink = edge

- Key point

- ✓ GNN은 관계 구조를 가진 데이터를 학습하기 위한 딥러닝 모델
- ✓ 개별 객체뿐 아니라, 객체들이 어떻게 연결되어 있는지를 함께 학습



02 | Graph Basics

Node, Edge, Feature, Adjacency Matrix

• Definition

- ✓ Graph는 $G = (V, E)$ 로 표현
- ✓ V : node 또는 vertex의 집합
- ✓ E : node 사이 edge 또는 link의 집합

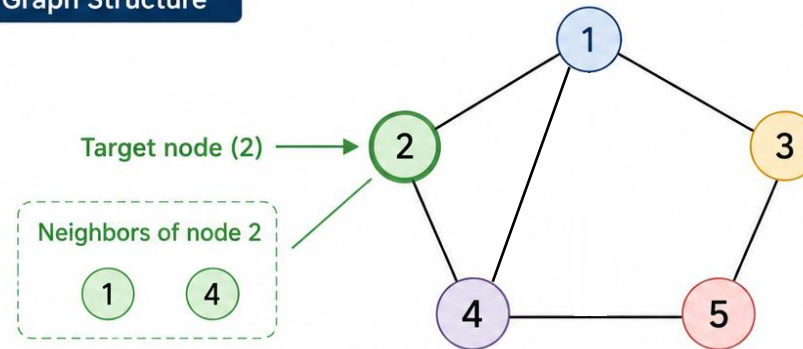
• Components

- ✓ Node: 그래프를 구성하는 대상
- ✓ Edge: node 사이의 연결 관계
- ✓ Neighbor: 특정 node와 edge로 연결된 주변 node
- ✓ Node feature: 각 node가 가진 속성 벡터 x_n

• Adjacency Matrix

- ✓ 그래프의 연결 구조를 행렬 A 로 표현
- ✓ edge가 있으면 1, 없으면 0
- ✓ undirected graph에서는 A 가 대칭 행렬

Graph Structure



Node Features

1	$x_1 = [1, 0, 2]$
2	$x_2 = [0, 1, 1]$
3	$x_3 = [1, 1, 0]$
4	$x_4 = [0, 2, 1]$
5	$x_5 = [1, 0, 1]$

Adjacency Matrix

- $A \in \mathbb{R}^{N \times N}$
- $A_{ij} = 1$ if there is an edge between node i and j
- Undirected graph $\Rightarrow A$ is symmetric

	1	2	3	4	5
1	0	1	1	1	0
2	1	0	0	1	0
3	1	0	0	0	1
4	1	1	0	0	1
5	0	0	1	1	0

1 : edge exists
0 : no edge

Graph = Structure (A) + Node Features (X)

03 | Prediction Tasks on Graphs

Node, Edge, and Graph-Level Prediction

- Prediction Targets

- ✓ Node prediction: 각 node의 label 예측
- ✓ Edge prediction: 두 node 사이의 관계 예측
- ✓ Graph prediction: 전체 graph의 성질 예측

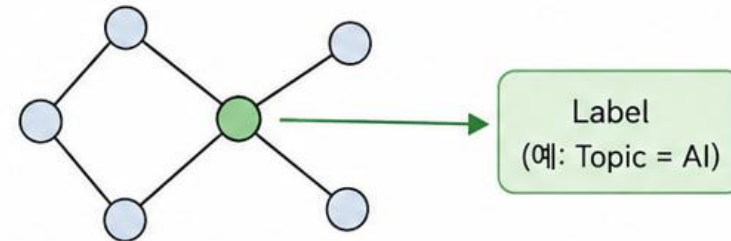
- Examples

- ✓ Node: 문서의 topic 분류
- ✓ Edge: social network에서 친구 추천
- ✓ Graph: molecule의 solubility / toxicity 예측

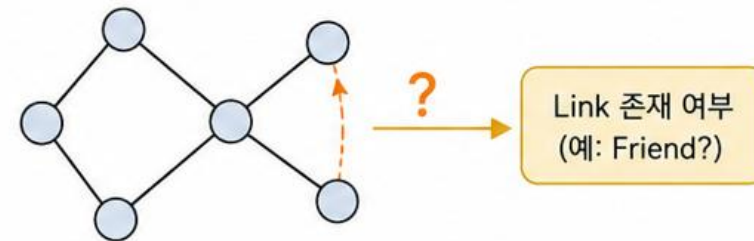
- Key point

- ✓ GNN은 예측 대상에 따라 readout 방식이 달라짐
- ✓ 같은 graph embedding 구조를 다양한 문제에 활용 가능

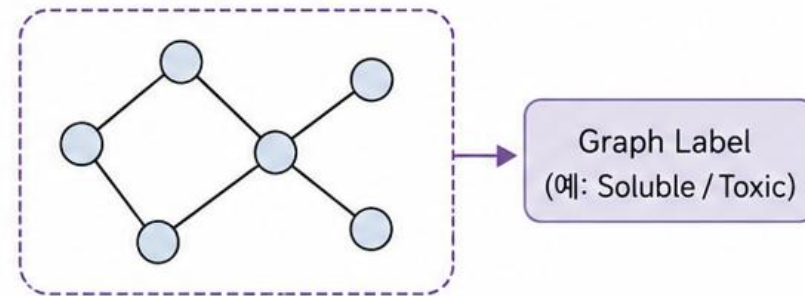
1) Node Prediction



2) Edge Prediction



3) Graph Prediction



04 | Node Ordering Problem

Why Node Order Matters in Graphs

- Core idea

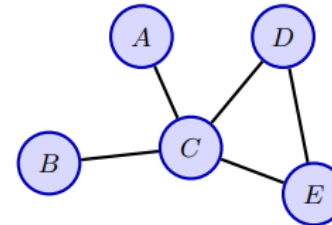
- ✓ 그래프를 adjacency matrix로 표현하려면 node 순서를 정해야 함
- ✓ 하지만 node ordering은 사람이 임의로 정하는 값
- ✓ 같은 graph라도 node 순서가 바뀌면 matrix 형태가 달라짐

- Permutation in Math

- ✓ 노드 순서 바꾸기를 수학적으로 다루려면 *permutation matrix* P 도입
- ✓ P : node ordering을 바꾸는 permutation matrix
- ✓ 노드 순서가 바뀔 때: $\tilde{X} = PX$, $\tilde{A} = PAP^T$

- Key Point

- ✓ $\tilde{X}, \tilde{A}$ 로 행렬이 달라졌다고 graph의 의미가 달라지는 것은 아님
- ✓ 모델은 matrix 배열이 아니라 graph structure를 학습해야 함
- ✓ 따라서 모델은 permutation invariance / equivariance 성질이 필요



(a)

	A	B	C	D	E
A			1		
B			1		
C	1	1		1	1
D			1		1
E			1	1	

(b)

	C	E	A	D	B
C		1	1	1	
E	1			1	
A	1				
D	1	1			
B	1				

(c)

05 | Invariance vs Equivariance

Permutation Invariance and Equivariance

- Graph-level Prediction

- ✓ node 순서가 바뀌어도 전체 graph의 예측값은 같아야 함
- ✓ 예: 같은 분자는 원자 순서를 바꿔도 solubility 예측이 같아야 함
- ✓ 이를 **Permutation Invariance**라고 함 $y(\tilde{X}, \tilde{A}) = y(X, A)$

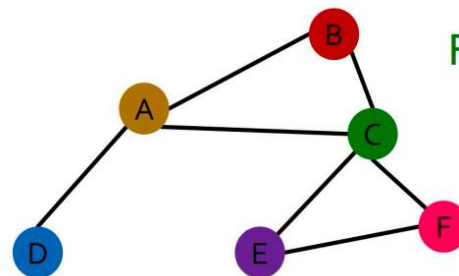
- Node-level Prediction

- ✓ node 순서가 바뀌면 node별 output 순서도 같이 바뀌어야 함
- ✓ 이를 **Permutation Equivariance**라고 함 $y(\tilde{X}, \tilde{A}) = Py(X, A)$

- Key point

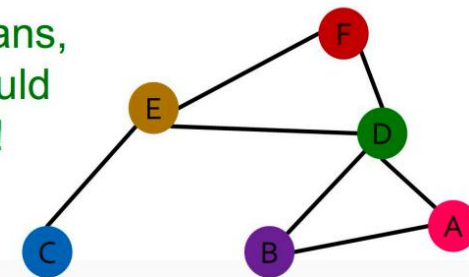
- ✓ 따라서 GNN은 기본적으로 노드 수준에서 **equivariance**를 유지하고, graph-level prediction이 필요할 때 마지막 readout에서 **invariance**를 얻음

Order plan 1: A_1, X_1

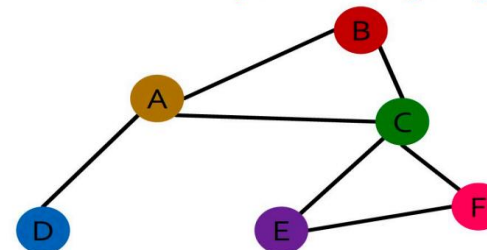


For two order plans,
output of f should
be the same!

Order plan 2: A_2, X_2



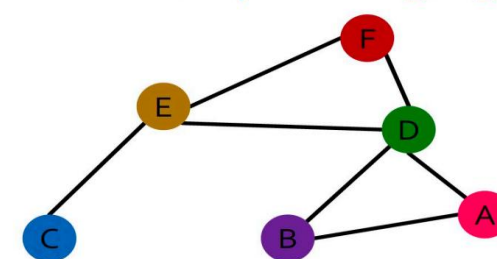
Order plan 1: A_1, X_1



$f(A_1, X_1) =$

A	yellow	blue
B	red	red
C	green	green
D	blue	blue
E	purple	purple
F	pink	pink

Order plan 2: A_2, X_2



$f(A_2, X_2) =$

A	red	red
B	purple	purple
C	blue	blue
D	green	green
E	yellow	yellow
F	pink	pink

06 | Neural Message Passing

Core Mechanism of Graph Neural Networks

• Core Idea

- ✓ Equivariant한 layer를 만드는 두 가지 조건
 1. 이웃 정보는 순서에 무관한 연산 (sum/mean/max)으로 모음
 2. 모든 노드에 같은 함수를 적용
- ✓ 이 두 조건을 만족시키는 방식이 Neural Message Passing
- ✓ 초기 node embedding은 node feature에서 시작함 $h_n^{(0)} = x_n$

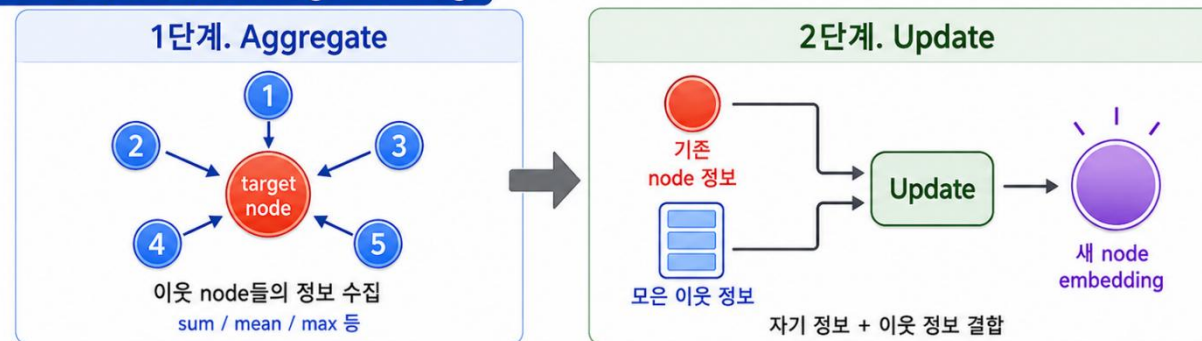
• Message Passing

- ✓ 1단계: 이웃 node들의 embedding을 모아 aggregate
- ✓ 2단계: 기존 node embedding과 message를 결합해 update
- ✓ $z_n^{(l)} = \text{Aggregate}(\{h_m^{(l)} : m \in N(n)\})$
- ✓ $h_n^{(l+1)} = \text{Update}(h_n^{(l)}, z_n^{(l)})$

• Key Point

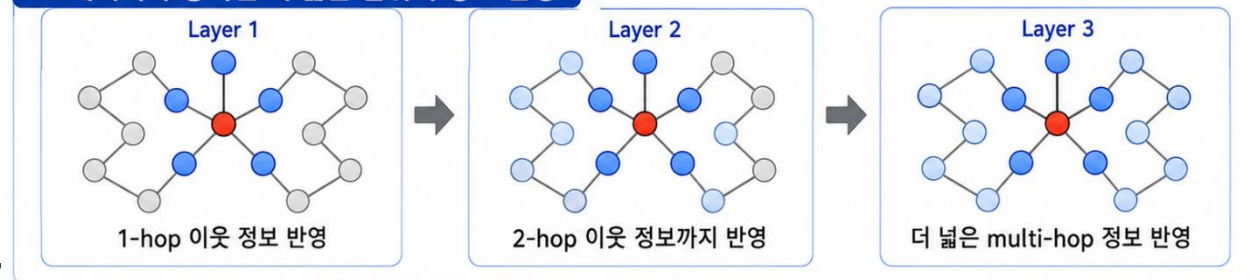
- ✓ Aggregate가 순서 무관 연산이고 모든 노드가 같은 함수를 쓰니까
→ 자동으로 permutation equivariant
- ✓ layer가 쌓일수록 더 넓은 범위의 node 정보를 반영

1. 한 레이어의 Message Passing



💡 한 레이어 = 이웃 정보 모으기(Aggregate) → node 표현 갱신(Update)

2. 레이어가 쌓이면 더 넓은 범위의 정보 반영



07 | Aggregation & Update

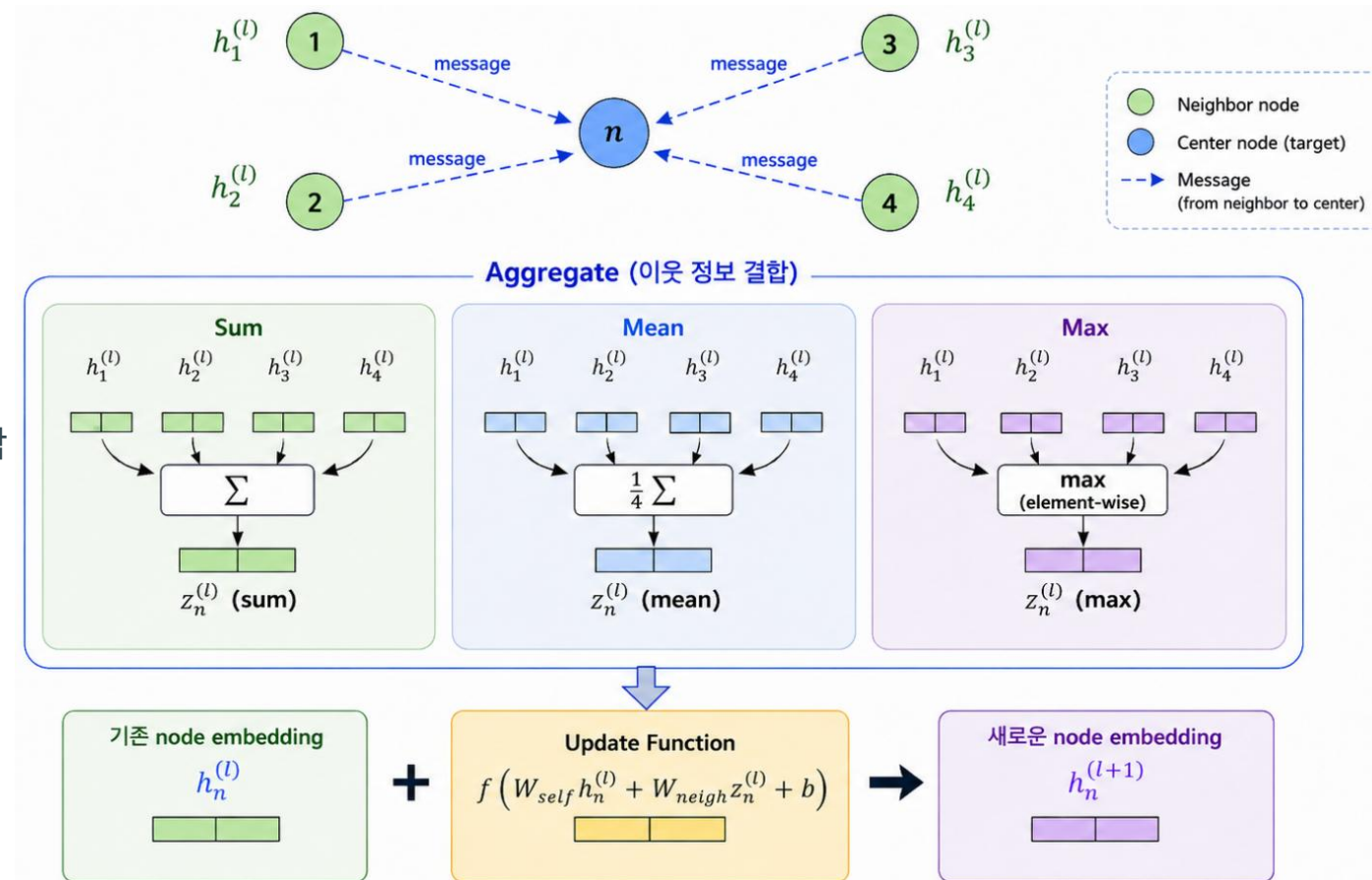
How Messages Are Combined

- Aggregation Operators

- ✓ Sum: 이웃 정보를 모두 더함 $z_n^{(l)} = \sum_{m \in N(n)} h_m^{(l)}$
- ✓ Mean: 이웃 수로 나누어 평균화 $z_n^{(l)} = \frac{1}{|N(n)|} \sum_{m \in N(n)} h_m^{(l)}$
- ✓ Max: 가장 강한 feature만 선택

- Update Function

- ✓ aggregated message $z_n^{(l)}$ 를 기존 node embedding과 결합
- ✓ MLP, linear layer, activation function 등을 사용
- ✓ 최종적으로 다음 layer embedding 생성
- ✓ $h_n^{(l+1)} = f(W_{self}h_n^{(l)} + W_{neigh}z_n^{(l)} + b)$



08 | Prediction Tasks with GNN

Node, Edge, and Graph Readout

• Node Classification

- ✓ 최종 node embedding $h_n^{(L)}$ 으로 node label 예측

$$y_{ni} = \frac{\exp(w_i^T h_n^{(L)})}{\sum_j \exp(w_j^T h_n^{(L)})}$$

• Edge Prediction

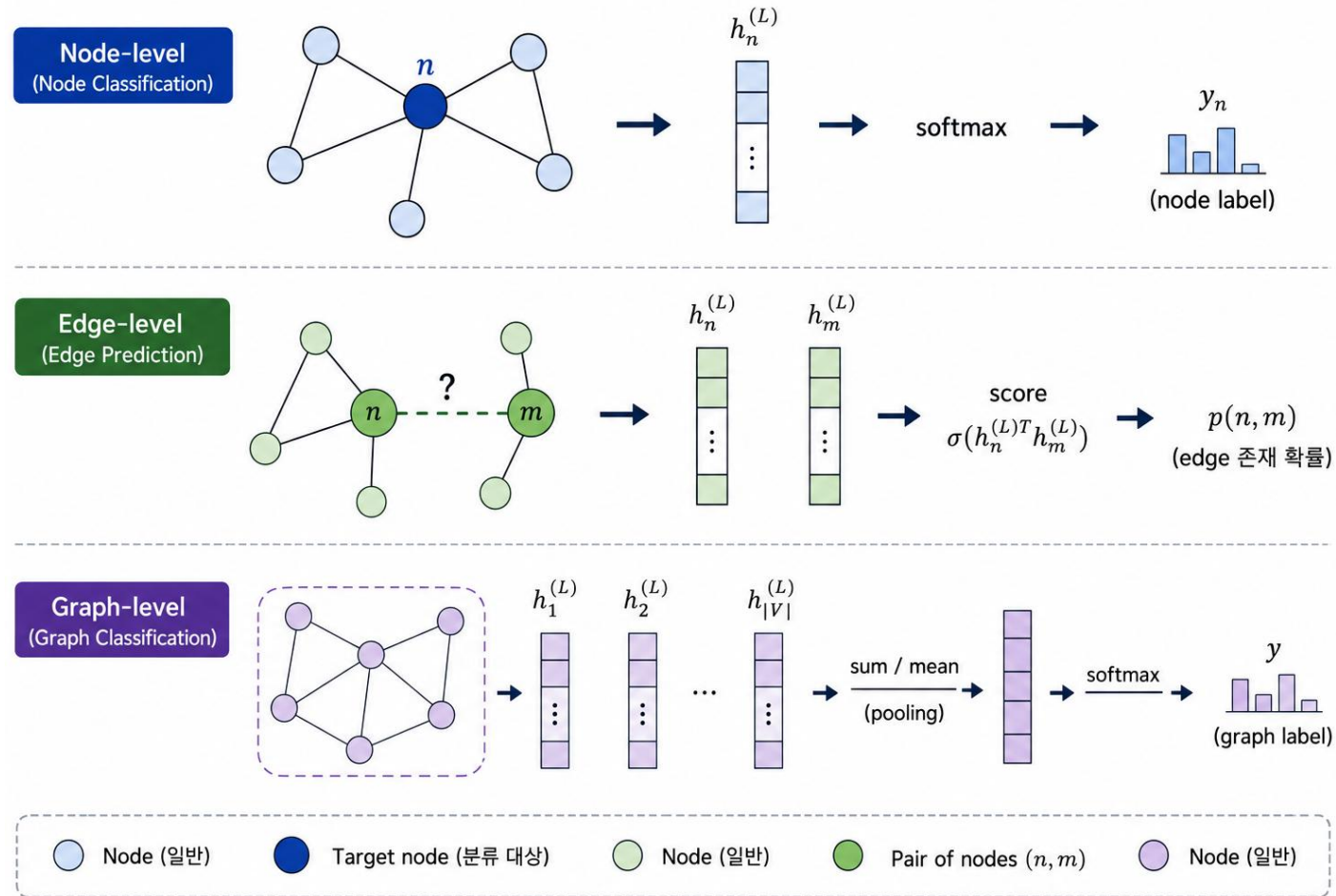
- ✓ 두 node embedding의 관계로 edge 존재 가능성 예측

$$p(n, m) = \sigma(h_n^T h_m)$$

• Graph Classification

- ✓ 전체 node embedding을 pooling해서 graph label 예측

$$y = f\left(\sum_{n \in V} h_n^{(L)}\right) \quad f = \text{softmax}(Wg + b)$$



09 | Graph Attention Networks

Learning Which Neighbors Matter

- Core Idea

- ✓ 중요한 *neighbor*의 정보는 크게, 덜 중요한 *neighbor*의 정보는 작게 반영
- ✓ GAT는 neighbor message에 attention coefficient A_{nm} 을 부여

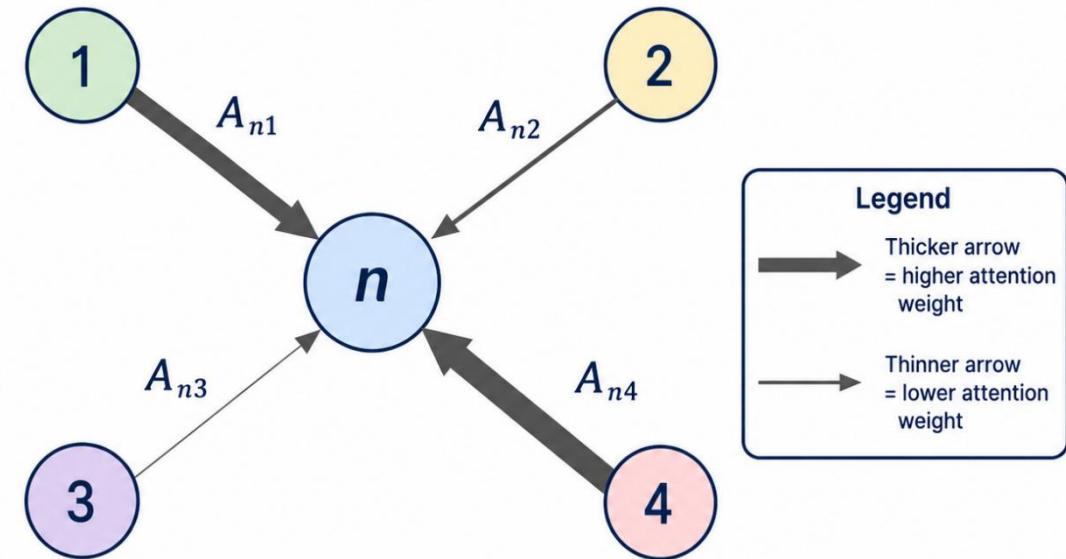
- Attention Aggregation

- ✓ $z_n^{(l)} = \sum_{m \in N(n)} A_{nm} h_m^{(l)}$
- ✓ $A_{nm} > 0, \sum_{m \in N(n)} A_{nm} = 1$

- Attention Weight

- ✓
$$A_{nm} = \frac{\exp(h_n^T W h_m)}{\sum_{m' \in N(n)} \exp(h_n^T W h_{m'})}$$

Attention weights control message importance



$$z_n^{(l)} = \sum_{m \in N(n)} A_{nm} h_m^{(l)}$$

10 | Edge & Graph Embeddings

Beyond Node Embeddings — General Framework

- Core Idea

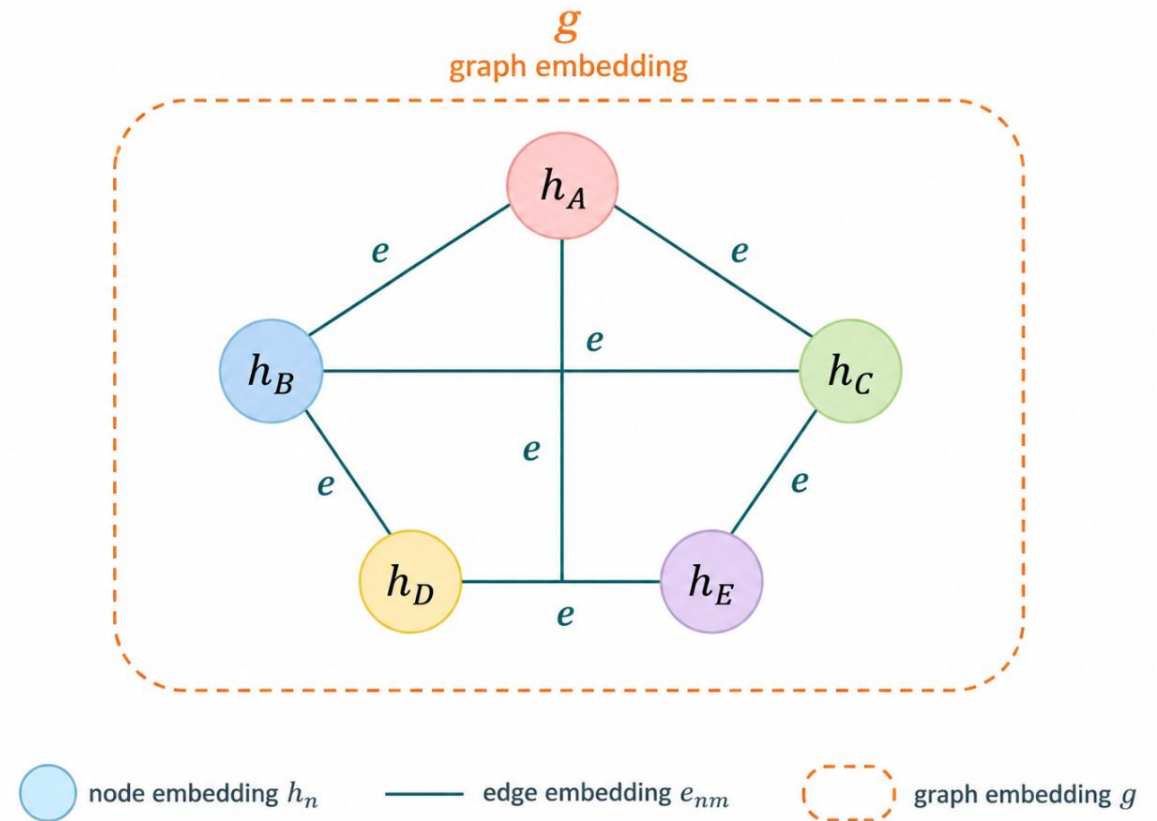
- ✓ 지금까지는 node embedding 만 다룸
- ✓ edge와 graph 전체에도 embedding 부여 가능

- Edge Embeddings

- ✓ edge e_{nm} 에 별도 embedding 부여
- ✓ $e_{nm}^{(l+1)} = \text{Update}_{\text{edge}}(e_{nm}^{(l)}, h_n^{(l)}, h_m^{(l)})$
- ✓ 활용: 분자 결합 종류

- Graph Embeddings

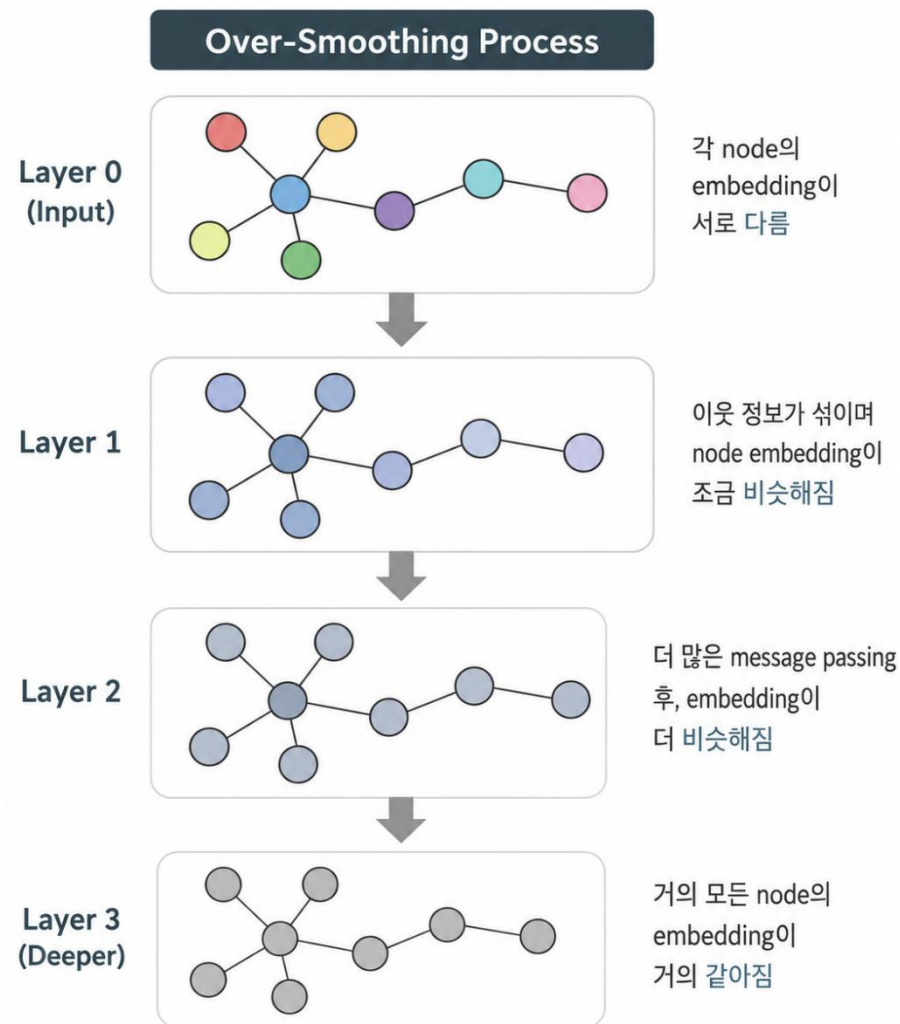
- ✓ 그래프 전체 embedding g
- ✓ $g^{(l+1)} = \text{Update}_{\text{graph}}(g^{(l)}, \{h_n^{(l)}\}, \{e_{nm}^{(l)}\})$
- ✓ 단순 message passing은 이 framework의 특수 경우



11 | Challenges

Over-Smoothing

- Over-Smoothing
 - ✓ message passing을 여러 번 반복하면 node embedding들이 서로 비슷해질 수 있음
 - ✓ layer가 깊어질수록 node별 구분력이 약해질 수 있음
- How to Alleviate
 - ✓ 1. residual connection으로 이전 layer 정보를 보존
 - ✓ $h_n^{(l+1)} = \text{Update}(h_n^{(l)}, z_n^{(l)}) + h_n^{(l)}$
 - ✓ 2. 마지막 layer만 사용하지 않고, 여러 layer의 embedding을 함께 활용
 - ✓ $y_n = f(h_n^{(1)} \oplus h_n^{(2)} \oplus \dots \oplus h_n^{(L)})$



Q&A

감사합니다.